

ASTON.

Обобщения



astondevs.ru

Понятие обобщения



Представим что перед нами поставлена задача создать класс, который позволит хранить в себе один объект любого типа. В таком случае мы можем создать класс SimpleBox, у которого будет единственное поле типа Object, в которое можно будет записать объект абсолютно любого типа. Далее для проверки работы нашего класса напишем немного кода в классе BoxDemoApp.

В первых строках кода метода main() мы создаем две коробки intBox1 и intBox2, в которые при инициализации складываем значения 20 и 30. Представим что нам надо вытащить из коробок числа и получить их сумму.

Для того чтобы вытащить из коробки число можно применить метод getObj(), но он вернет нам не int а Object, следовательно, нужно еще добавить приведение к типу Integer. Во-вторых, чтобы не получить ClassCastException, перед строками с приведением типов желательно делать проверку (instanceof) на то, что в коробках лежит именно то, что мы ожидаем. В результате мы получим сумму чисел из коробок и напечатаем ее в консоль.

```
public class SimpleBox {
    private Object obj;
    public Object getObj() {
        return obj;
    }
    public void setObj(Object obj) {
        this.obj = obj;
    }
    public SimpleBox(Object obj) {
        this.obj = obj;
    }
}

public class BoxDemoApp {
    public static void main(String[] args) {
        SimpleBox intBox1 = new SimpleBox(20);
        SimpleBox intBox2 = new SimpleBox(30);
        if (intBox1.getObj() instanceof Integer && intBox2.getObj()
instanceof Integer) {
            int sum = (Integer)intBox1.getObj() +
(Integer)intBox2.getObj();
            System.out.println("sum = " + sum);
        } else {
            System.out.println("Содержимое коробок отличается по
типу");
        }
        // вызвали какой-нибудь метод, которому отдали intBox1
        // и этот метод кладет в коробку String
        intBox1.setObj("Java");
        // продолжаем наш код, и при выполнении получим ClassCastException
        int secondSum = (Integer)intBox1.getObj() +
(Integer)intBox2.getObj();
    }
}
```

Понятие обобщения



Best practice: проверку на `instanceOf` из примера лучше не использовать в промышленном коде. Предположим вы отфильтровали в коллекции только элементы `Object`, которые являются `Integer`.

Чтобы ничего не взорвалось с `ClassCastException`. Но тогда, если другой пользователь вашей коробки положит туда строку "1000" - вы потеряете это значение. Правильнее сделать коробку типизированной.

Эти две особенности использования обобщений и приводят к повышению безопасности типов данных, и упрощению работы при написании кода.

Представим что в коде мы вызываем некий метод, и отдаем туда ссылку на `intBox1`. Выполняемый метод может положить в нашу коробку все что угодно, например, строку `Java`. И если после этого, мы попытаемся сложить содержимое наших коробок, забыв при этом сделать `instanceof`, то получим `ClassCastException`.

Понятие обобщения



Подводя итог, можно выделить три проблемы при использовании такого подхода:

- Каждый раз, когда мы хотим вытащить данные из нашей универсальной коробки, нам необходимо выполнять приведение типов;
- Чтобы не получить `ClassCastException`, перед каждым приведением типов, необходимо делать проверку типов данных с помощью `instanceof`;
- Если мы где-то будем применять приведение типов, и забудем прописать `instanceof`, то появится вероятность появления `ClassCastException` в этой части кода.

Понятие обобщения

Для того, чтобы решить эти проблемы, в Java применяются обобщения.

Обобщения – это параметризованные типы, которые позволяют объявлять обобщенные классы, интерфейсы и методы, где тип данных, которыми они оперируют, указан в виде параметра. Обобщения добавляют в Java безопасность типов и делают управление проще.

Исключается необходимость применять явные приведения типов, так как благодаря обобщениям все приведения выполняются неявно, в автоматическом режиме.

Чтобы понять почему нам больше не надо делать явное приведение типов, и откуда берется какая-то безопасность, рассмотрим простой пример обобщенного класса.



```
public class GenericBox<T> {
    private T obj;
    public GenericBox (T obj) {
        this.obj = obj;
    }
    public T getObj () {
        return obj;
    }
    public void showType () {
        System.out.println ("Тип T: " +
obj.getClass ().getName ());
    }
}

public class GenericsDemoApp {
    public static void main (String args[]) {
        GenericBox<String> genStr = new
GenericBox<> ("Hello");
        genStr.showType ();
        System.out.println ("genStr.getObject (): " +
genStr.getObj ());
        GenericBox<Integer> genInt = new GenericBox<> (140);
        genInt.showType ();
        System.out.println ("genInt.getObject (): " +
genInt.getObj ());
        int valueFromGenInt = genInt.getObj ();
        String valueFromGenString = getStr.getObj ();
        // genInt.setObj ("Java"); // Ошибка компиляции!!!
    }
}
```



Понятие обобщения



ВВ: Важно!

- При получении значений из `genInt` и `genStr` не требуется преобразование типов, `genInt.getObj()` сразу возвращает `Integer`, а `genStr.getObj()` - `String`. То есть приведение типов выполняется неявно и автоматически.
- Если объект создан как `GenericBox genInt`, то мы не сможем записать в него строку: `genInt.setObj("Java")`. При попытке написать такую строку кода, мы получим ошибку на этапе компиляции. То есть обобщения отслеживают корректность используемых типов данных.

Эти две особенности использования обобщений и приводят к повышению безопасности типов данных, и упрощению работы при написании кода.

Результат выполнения:

```
Тип T: java.lang.String
genStr.getObj(): Hello
Тип T: java.lang.Integer
genInt.getObj(): 140
```

В объявлении класса `public class GenericBox`, `T` представляет собой имя параметра типа, на место которого при создании объекта класса `GenericBox` будет подставлен конкретный тип данных. У объекта `genStr` из примера выше `T = String`, а для `genInt` `T = Integer`. То есть используя обобщения мы можем создавать переменные типы данных.

Понятие обобщения



NB: Важно! Обобщения работают только с ссылочными типами данных. Для работы с примитивами надо будет использовать классы-обертки.

Буква T в объявлении обобщенного класса не является обязательной, вы можете вместо нее использовать любую другую букву (E, N, V, ...), это всего лишь имя переменного типа. Ссылка на одну специфическую версию обобщенного типа не обладает совместимостью с другой версией того же обобщенного типа. Например:

```
GenericBox<String> genStr = new  
GenericBox<>("Hello");  
GenericBox<Integer> genInt = new GenericBox<>(140);  
genInt = genStr; // Ошибка
```

Объекты genInt и genStr принадлежат классу GenericBox, но представляют собой ссылки на разные типы – ведь типы их параметров отличаются.

Обобщённые классы, интерфейсы, методы



Для обобщенного типа можно объявлять более одного параметра, используя список, разделенный запятыми:

```
public class TwoGen<T, V> {  
  
    private T obj1;  
    private V obj2;  
    public TwoGen(T obj1, V obj2) {  
        this.obj1 = obj1;  
        this.obj2 = obj2;  
    }  
    public void showTypes() {  
        System.out.println("Тип T: " + obj1.getClass().getName());  
        System.out.println("Тип V: " + obj2.getClass().getName());  
    }  
    public T getObj1() {  
        return obj1;  
    }  
    public V getObj2() {  
        return obj2;  
    }  
}  
  
public class SimpleGenApp {  
    public static void main(String args[]) {  
        TwoGen<Integer, String> twoGenObj = new TwoGen<Integer,  
String>(555, "Hello");  
        twoGenObj.showTypes();  
        int intValue = twoGenObj.getObj1();  
        String strValue = twoGenObj.getObj2();  
        System.out.println(intValue);  
        System.out.println(strValue);  
    }  
}
```


Ограниченные типы



В примерах, рассмотренных выше, параметры типов можно было заменить любыми типами классов. Но иногда существует необходимость ограничить набор этих перечень типов.

Создадим обобщенный класс, который содержит в себе массив (мы предполагаем что это будет массивом чисел любого типа) и метод, возвращающий среднее значение этого массива.

```
public class BoxWithNumbers<T> {  
    private T[] nums;  
    public BoxWithNumbers(T... nums) {  
        this.nums = nums;  
    }  
    public double avg() {  
        double sum = 0.0;  
        for(int i = 0; i < nums.length; i++) {  
            sum += nums[i]; // Ошибка  
        }  
        return sum / nums.length;  
    }  
}
```

Ограниченные типы



В методе `avg()` мы создаем временную `double` переменную `sum` и пытаемся с помощью нее сложить все числа, лежащие в массиве `nums`. Но первая проблема заключается в том, что Java видит что мы пытаемся складывать объекты (хотя мы собираемся хранить в этом массиве только числа), и конечно же выдает ошибку. Вторая же проблема заключается в том, что если даже это будет массив `Integer`, то у каждого объекта при суммировании надо будет вызывать метод `doubleValue()`, который будет преобразовывать каждый объект `Integer` в примитив `double`. Давайте попробуем разобраться с двумя этими проблемами.

При работе с обобщениями будем использовать ограниченные типы. Когда указывается параметр типа, можно создать ограничение сверху, которое укажет суперкласс, от которого должны быть унаследованы все аргументы типов. Для этого используется ключевое слово `extends`:

```
<T extends суперкласс>
```

Ограниченные типы

NB: Важно!

В роли ограничителя сверху может выступать не только класс, но и один или несколько интерфейсов. Для указания нескольких элементов используется оператор &.

Обратите внимание, что даже если вы ограничиваете T интерфейсом, все равно используется ключевое слово `extends`. Если в качестве ограничителя используется класс и интерфейс, то класс должен быть указан первым. Это означает, что параметр T может быть заменен только самим суперклассом или его подклассами. Он объявляет включающую верхнюю границу. Можно использовать ограничение сверху, чтобы исправить класс `BoxWithNumbers`, указав класс `Number` как верхнюю границу используемого параметра типа. Посмотрим что нам это даст.

Объявление `public class BoxWithNumbers` сообщает компилятору, что все объекты типа T являются подклассами класса `Number`, и поэтому могут вызывать метод `doubleValue()`, как и любой другой из класса `Number`. Ограничивая параметр T, мы предотвращаем создание нечисловых объектов класса `BoxWithNumbers`.



```
public class BoxWithNumbers<T extends Number> {

    private T[] nums;

    public BoxWithNumbers(T... nums) {
        this.nums = nums;
    }

    public double avg() {
        double sum = 0.0;
        for(int i = 0; i < nums.length; i++) {
            // У nums[i] появился метод doubleValue() из класса Number
            // который позволяет любой числовой объект привести к double
            sum += nums[i].doubleValue();
        }
        return sum / nums.length;
    }
}

public class BoxWithNumbersDemoApp {

    public static void main(String args[]) {
        BoxWithNumbers<Integer> intBoxWithNumbers = new
BoxWithNumbers<Integer>(1, 2, 3, 4, 5);
        System.out.println("Ср. знач. intBoxWithNumbers равно " +
intBoxWithNumbers.avg());

        BoxWithNumbers<Double> doubleBoxWithNumbers = new
BoxWithNumbers<Double>(1.0, 2.0, 3.0, 4.0, 5.0);
        System.out.println("Ср. знач. doubleBoxWithNumbers равно " +
doubleBoxWithNumbers.avg());
        // Это не скомпилируется, потому что String не является подклассом
        Number
        // BoxWithNumbers<String> strBoxWithNumbers = new
        BoxWithNumbers<>("1", "2", "3", "4", "5");
        // System.out.println("Ср. знач. strBoxWithNumbers равно " +
        strBoxWithNumbers.avg());
    }
}
```

Ограниченные типы



Объявление `public class BoxWithNumbers` сообщает компилятору, что все объекты типа `T` являются подклассами класса `Number`, и поэтому могут вызывать метод `doubleValue()`, как и любой другой из класса `Number`. Ограничивая параметр `T`, мы предотвращаем создание нечисловых объектов класса `BoxWithNumbers`.

Использование метасимвольных аргументов



Давайте попробуем добавить в класс `BoxWithNumbers` метод `sameAvg()`, который будет проверять равенство средних значений массивов двух объектов типа `BoxWithNumbers`, независимо от того, какого типа числовые значения в них содержатся. Допустим в одном будет `new int[] { 1, 2, 3 }`, а во втором `new double { 1.0, 2.0, 3.0 }`; Метод `sameAvg()` на вход должен принимать объект типа `BoxWithNumbers` и сравнивать его среднее значение со средним значением вызывающего объекта. Код применения этого метода может выглядеть вот так:

```
public class BoxWithNumbersDemoApp {  
    public static void main(String args[]) {  
        BoxWithNumbers<Integer> intBoxWithNumbers = new  
BoxWithNumbers<>(1, 2, 3, 4, 5);  
        BoxWithNumbers<Double> doubleBoxWithNumbers = new  
BoxWithNumbers<>(1.0, 2.0, 3.0, 4.0, 5.0);  
        if (intBoxWithNumbers.sameAvg(doubleBoxWithNumbers))  
        {  
            System.out.println("Средние значения равны");  
        } else {  
            System.out.println("Средние значения не равны");  
        }  
    }  
}
```


Использование метасимвольных аргументов



NB: Заметка. Чтобы не столкнуться с ошибкой округления при сравнении двух дробных чисел, мы сравниваем средние значения в пределах дельты 0.0001

Такой код будет работать только с объектом класса BoxWithNumbers, тип которого совпадает с вызывающим объектом. Если вызывающий объект имеет тип BoxWithNumbers, то параметр another обязательно должен принадлежать к аналогичному типу

```
public class BoxWithNumbersDemoApp {  
    public static void main(String args[]) {  
        BoxWithNumbers<Integer> intBoxWithNumbers1 = new  
BoxWithNumbers<>(1, 2, 3, 4, 5);  
        BoxWithNumbers<Integer> intBoxWithNumbers2 = new  
BoxWithNumbers<>(2, 1, 3, 4, 5);  
        BoxWithNumbers<Double> doubleBoxWithNumbers = new  
BoxWithNumbers<>(1.0, 2.0, 3.0, 4.0, 5.0);  
  
        System.out.println(intBoxWithNumbers1.sameAvg(intBoxWithN  
umbers2));  
        //Так работает  
  
        System.out.println(intBoxWithNumbers1.sameAvg(doubleBoxWi  
thNumbers));  
        // Ошибка  
        // (T = Integer) != (T = Double)  
    }  
}
```

```
public class BoxWithNumbers<T extends Number> {  
    // ...  
    public boolean sameAvg(BoxWithNumbers<T> another) {  
        return Math.abs(this.avg() - another.avg()) <  
0.0001;  
    }  
    // ...  
}
```

Использование метасимвольных аргументов



Чтобы создать обобщенную версию метода `sameAvg()`, следует использовать метасимвольные аргументы. Это средство обобщений Java, которое обозначается символом «?» и представляет собой неизвестный тип.

```
public class BoxWithNumbers<T extends Number> {  
    // ...  
    public boolean sameAvg(BoxWithNumbers<?> another) {  
        return Math.abs(this.avg() - another.avg()) < 0.0001;  
    }  
    // ...  
}
```

`BoxWithNumbers` соответствует любому объекту класса `BoxWithNumbers`. Можно сравнивать средние значения любых двух объектов этого класса.

Результат работы программы:

```
Среднее iBoxWithNumbers = 3.0  
Среднее dBoxWithNumbers = 3.3  
Среднее fBoxWithNumbers = 3.0  
Средние iBoxWithNumbers и dBoxWithNumbers  
отличаются  
Средние iBoxWithNumbers и fBoxWithNumbers равны
```

Метасимвольный аргумент не влияет на тип создаваемого объекта класса `BoxWithNumbers`. Это зависит от `extends` в объявлении класса `BoxWithNumbers`.

```
public class BoxWithNumbers<T extends Number> {  
  
    private T[] nums;  
    public BoxWithNumbers(T[] nums) {  
        this.nums = nums;  
    }  
    public double avg() {  
        double sum = 0.0;  
        for (int i = 0; i < nums.length; i++) {  
            sum += nums[i].doubleValue();  
        }  
        return sum / nums.length;  
    }  
    public boolean sameAvg(BoxWithNumbers<?> another) {  
        return Math.abs(this.avg() - another.avg()) < 0.0001;  
    }  
}  
  
public class WildcardDemoApp {  
  
    public static void main(String args[]) {  
        BoxWithNumbers<Integer> iBoxWithNumbers = new BoxWithNumbers<>(1, 2, 3, 4, 5);  
        System.out.println("Среднее iBoxWithNumbers = " + iBoxWithNumbers.avg());  
        BoxWithNumbers<Double> dBoxWithNumbers = new BoxWithNumbers<>(1.1, 2.2, 3.3, 4.4,  
5.5);  
        System.out.println("Среднее dBoxWithNumbers = " + dBoxWithNumbers.avg());  
        BoxWithNumbers<Float> fBoxWithNumbers = new BoxWithNumbers<>(1.0f, 2.0f, 3.0f, 4.0f,  
5.0f);  
        System.out.println("Среднее fBoxWithNumbers = " + fBoxWithNumbers.avg());  
        System.out.print("Средние iBoxWithNumbers и dBoxWithNumbers ");  
        if (iBoxWithNumbers.sameAvg(dBoxWithNumbers)) {  
            System.out.println("равны");  
        } else {  
            System.out.println("отличаются");  
        }  
        System.out.print("Средние iBoxWithNumbers и fBoxWithNumbers");  
        if (iBoxWithNumbers.sameAvg(fBoxWithNumbers)) {  
            System.out.println("равны");  
        } else {  
            System.out.println("отличаются");  
        }  
    }  
}
```

Ограничения при использовании обобщений



Ограничения на статические члены

Никакой статический член не может использовать тип параметра, объявленный в его классе.

```
public class WrongGenericClass<T> {  
  
    static T data; // Неверно, нельзя создать статические  
    переменные типа T  
    static T getData () {  
        return data;  
    } // Неверно, ни один статический метод не может  
    использовать T  
}
```

Нельзя объявить статические члены, использующие обобщенный тип.
Но можно объявлять обобщенные статические методы, определяющие их собственные параметры типа.

Ограничения обобщенных исключений

Обобщенный класс не может расширять класс Throwable. Значит, создать обобщенные классы исключений невозможно.

Практическое задание



Необходимо написать программу, позволяющую выполнить следующее:

1. Даны классы Fruit, Apple extends Fruit, Orange extends Fruit;
2. Класс Box, в который можно складывать фрукты. Коробки условно сортируются по типу фрукта, поэтому в одну коробку нельзя сложить и яблоки, и апельсины;
3. Для хранения фруктов внутри коробки можно использовать ArrayList;
4. Сделать метод `getWeight()`, который высчитывает вес коробки, зная вес одного фрукта и их количество: вес яблока – 1.0f, апельсина – 1.5f (единицы измерения не важны);

5. Внутри класса Box сделать метод `compare()`, который позволяет сравнить текущую коробку с той, которую подадут в `compare()` в качестве параметра. `true` – если их массы равны, `false` в противном случае. Можно сравнивать коробки с яблоками и апельсинами;
6. Написать метод, который позволяет пересыпать фрукты из текущей коробки в другую. Помним про сортировку фруктов: нельзя яблоки высыпать в коробку с апельсинами. Соответственно, в текущей коробке фруктов не остается, а в другую перекидываются объекты, которые были в первой;
7. Не забываем про метод добавления фрукта в коробку.